Spring Boot & REST APIs — Complete Theory Guide

Concepts • Real-World Examples • Interview Questions

WHAT THIS GUIDE COVERS

- Spring Framework fundamentals — IoC, DI, Beans
- Spring Boot — Auto-configuration, Starters, Application structure
- REST API Development — Controllers, HTTP Methods, Status Codes
- Request/Response Handling — @PathVariable, @RequestParam, @RequestBody
- Spring Data JPA — Entities, Repositories, CRUD operations
- Exception Handling — @ControllerAdvice, Custom Exceptions
- Validation — @Valid, Bean Validation
- Real-world CIGNA examples throughout

1. SPRING FRAMEWORK FUNDAMENTALS

What is Spring Framework?

Spring is a comprehensive framework for building enterprise Java applications. It provides infrastructure support so you can focus on your business logic instead of plumbing code.

Spring solves common problems: dependency management, database connections, transaction handling, security, and more.

Core Concepts

1.1 IoC (Inversion of Control)

Traditional approach: YOU create objects using 'new' keyword.

Spring approach: Spring CONTAINER creates and manages objects for you.

Example - Traditional Way:

```
// You create dependencies manually
PrescriptionRepository repo = new PrescriptionRepository();
PrescriptionService service = new PrescriptionService(repo);

// Problem: Tight coupling, hard to test, hard to change
```

Example - Spring Way (IoC):

```
@Service
public class PrescriptionService {
    // Spring creates and injects this automatically
    @Autowired
    private PrescriptionRepository repository;
}

// Spring manages the lifecycle and dependencies
```

Why IoC?

- Loose coupling — easier to change implementations
- Easier testing — can inject mock objects
- Centralized configuration
- Spring handles object lifecycle (creation, destruction)

1.2 Dependency Injection (DI)

Dependency Injection is HOW Spring implements IoC. Instead of you creating dependencies, Spring injects them.

Three Types of DI:

1. Constructor Injection (Recommended):

```
@Service
public class PrescriptionService {
```

```
    private final PrescriptionRepository repository;

    // Spring injects dependency through constructor
    public PrescriptionService(PrescriptionRepository repository) {
        this.repository = repository;
    }
}
```

2. Field Injection (Easy but not recommended for testing):

```
@Service
public class PrescriptionService {
    @Autowired  // Spring injects directly into field
    private PrescriptionRepository repository;
}
```

3. Setter Injection (Rarely used):

```
@Service
public class PrescriptionService {
    private PrescriptionRepository repository;

    @Autowired
    public void setRepository(PrescriptionRepository repository) {
        this.repository = repository;
    }
}
```

1.3 Spring Beans

A Bean is simply an object that is managed by the Spring container.
You tell Spring which classes should be beans using annotations.

Common Stereotype Annotations:

```
@Component   // Generic bean
@Service     // Business logic layer
@Repository  // Data access layer (talks to database)
@Controller  // Web layer (handles HTTP requests)
```

Real-world example from CIGNA:

```
@Repository  // Spring creates and manages this bean
public interface PrescriptionRepository extends JpaRepository<Prescription, String> {
    // Spring generates implementation automatically
}

@Service  // Business logic bean
public class PrescriptionService {
    private final PrescriptionRepository repository;

    // Spring injects the repository bean here
    public PrescriptionService(PrescriptionRepository repository) {
        this.repository = repository;
    }
}
```

Interview Questions

Q1: What is IoC?

Inversion of Control means the control of object creation and lifecycle is inverted from the programmer to the Spring container. Instead of you creating objects with 'new', Spring creates and manages them.

Q2: What is Dependency Injection?

Dependency Injection is a design pattern where dependencies are provided (injected) to an object rather than the object creating them itself. Spring injects dependencies automatically through constructors, fields, or setters.

Q3: What is the difference between @Component, @Service, and @Repository?

All three mark classes as Spring beans. The difference is semantic:

- @Component — Generic bean, any Spring-managed component
- @Service — Business logic layer, contains service methods
- @Repository — Data access layer, handles database operations

Using specific annotations makes code more readable and enables Spring to apply special behaviors (e.g., @Repository automatically translates database exceptions).

Q4: Which type of Dependency Injection is recommended and why?

Constructor injection is recommended because:

- Makes dependencies explicit and required
- Easier to test (can pass mocks in constructor)
- Enables immutability (final fields)
- Prevents NullPointerException (dependency must be provided)

2. SPRING BOOT — SIMPLIFYING SPRING

What is Spring Boot?

Spring Boot is a framework built on top of Spring that makes it easier to create production-ready applications quickly.

Think of it as Spring + Auto-Configuration + Embedded Server + Defaults.

Key Features

2.1 Auto-Configuration

Spring Boot automatically configures your application based on the dependencies you add.

Example: If you add MySQL dependency, Spring Boot automatically configures database connection, you just provide URL/username/password.

2.2 Starter Dependencies

Pre-packaged sets of dependencies for common use cases.

```
<!-- In pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <!-- Includes: Spring MVC, Tomcat, JSON -->
</dependency>

<dependency>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
    <!-- Includes: Hibernate, JPA, Transaction Management -->
</dependency>
```

2.3 Embedded Server

Spring Boot includes Tomcat server. No need to deploy WAR files to external server.

Just run: java -jar myapp.jar — and your API is live!

2.4 application.properties / application.yml

Configuration file where you set database URL, port, logging level, etc.

```
# application.properties
server.port=8080

# Database connection
spring.datasource.url=jdbc:mysql://localhost:3306/cigna_db
spring.datasource.username=root
spring.datasource.password=password

# JPA settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

2.5 Main Application Class

Every Spring Boot app has a main class with @SpringBootApplication.

```
@SpringBootApplication  // Combines @Configuration + @EnableAutoConfiguration +
@ComponentScan
public class PharmacyApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(PharmacyApiApplication.class, args);
    }
}
```

Interview Questions

Q5: What is Spring Boot?

Spring Boot is an extension of the Spring framework that simplifies application setup and development. It provides auto-configuration, starter dependencies, and embedded servers to create production-ready applications quickly.

Q6: What is the difference between Spring and Spring Boot?

Spring is a comprehensive framework for Java applications. Spring Boot builds on top of Spring and adds:

- Auto-configuration (less manual configuration)
- Starter dependencies (pre-packaged dependency sets)
- Embedded server (no need for external Tomcat)
- Production-ready features (health checks, metrics)

Q7: What does @SpringBootApplication do?

It's a convenience annotation that combines three annotations:

- @Configuration — Marks class as source of bean definitions
- @EnableAutoConfiguration — Enables auto-configuration
- @ComponentScan — Scans for components in current package and sub-packages

3. REST API DEVELOPMENT

What is REST?

REST (Representational State Transfer) is an architectural style for designing web services. RESTful APIs use HTTP methods to perform operations on resources.

HTTP Methods (CRUD Operations)

```
GET     → Retrieve data (Read)
POST    → Create new resource (Create)
PUT     → Update existing resource (Update)
DELETE  → Remove resource (Delete)
PATCH   → Partial update (rarely used)
```

Building REST Controllers

3.1 @RestController

Marks a class as a REST controller. Combines @Controller + @ResponseBody.

Methods return data directly as JSON/XML instead of view names.

Example — Prescription REST Controller:

```
@RestController
@RequestMapping("/api/prescriptions")  // Base path for all endpoints
public class PrescriptionController {

    @Autowired
    private PrescriptionService service;

    // Endpoints go here
}
```

3.2 GET — Retrieve Data

```
// GET /api/prescriptions — Get all prescriptions
@GetMapping
public List<Prescription> getAllPrescriptions() {
    return service.findAll();
}

// GET /api/prescriptions/{id} — Get prescription by ID
@GetMapping("/{id}")
public Prescription getPrescriptionById(@PathVariable String id) {
    return service.findById(id)
        .orElseThrow(() -> new PrescriptionNotFoundException(id));
}

// GET /api/prescriptions/search?status=ACTIVE — Search with query param
@GetMapping("/search")
public List<Prescription> searchByStatus(@RequestParam String status) {
    return service.findByStatus(status);
}
```

3.3 POST — Create New Resource

```
// POST /api/prescriptions — Create new prescription
@PostMapping
public ResponseEntity<Prescription> createPrescription(
        @RequestBody Prescription prescription) {

    Prescription saved = service.save(prescription);
    return ResponseEntity
        .status(HttpStatus.CREATED)  // 201 status
        .body(saved);
}
```

3.4 PUT — Update Existing Resource

```
// PUT /api/prescriptions/{id} — Update prescription
@PutMapping("/{id}")
public Prescription updatePrescription(
        @PathVariable String id,
        @RequestBody Prescription prescription) {

    return service.update(id, prescription);
}
```

3.5 DELETE — Remove Resource

```
// DELETE /api/prescriptions/{id} — Delete prescription
@DeleteMapping("/{id}")
public ResponseEntity<Void> deletePrescription(@PathVariable String id) {
    service.deleteById(id);
    return ResponseEntity.noContent().build();  // 204 status
}
```

Important Annotations

@PathVariable

Extracts value from URL path.

```
// URL: /api/prescriptions/RX123
@GetMapping("/{id}")
public Prescription get(@PathVariable String id) {
    // id = "RX123"
}
```

@RequestParam

Extracts value from query parameters.

```
// URL: /api/prescriptions/search?status=ACTIVE&limit=10
```

```
@GetMapping("/search")
public List<Prescription> search(
        @RequestParam String status,
        @RequestParam(defaultValue = "20") int limit) {
    // status = "ACTIVE", limit = 10
}
```

@RequestBody

Converts incoming JSON to Java object automatically.

```
// Request body: {"drugName": "Aspirin", "patientName": "John"}
@PostMapping
public Prescription create(@RequestBody Prescription prescription) {
    // Spring converts JSON to Prescription object automatically
    return service.save(prescription);
}
```

HTTP Status Codes — What They Mean

Every REST API should return appropriate status codes.

```
2xx — Success
200 OK          → Request succeeded, returning data
201 Created     → New resource created successfully
204 No Content  → Request succeeded, no data to return (e.g., DELETE)

4xx — Client Errors
400 Bad Request       → Invalid request format
401 Unauthorized      → Authentication required
403 Forbidden         → Authenticated but not allowed
404 Not Found         → Resource doesn't exist
409 Conflict          → Resource already exists

5xx — Server Errors
500 Internal Server Error → Something went wrong on server
503 Service Unavailable   → Server temporarily down
```

How to return specific status codes:

```
// Option 1: Using ResponseEntity
@PostMapping
public ResponseEntity<Prescription> create(@RequestBody Prescription rx) {
    Prescription saved = service.save(rx);
    return ResponseEntity.status(HttpStatus.CREATED).body(saved);
}

// Option 2: Using @ResponseStatus
@PostMapping
@ResponseStatus(HttpStatus.CREATED)  // Always returns 201
public Prescription create(@RequestBody Prescription rx) {
    return service.save(rx);
}
```

Real-World Example — CIGNA Prescription API

Complete controller with all CRUD operations:

```
@RestController
@RequestMapping("/api/prescriptions")
public class PrescriptionController {

    private final PrescriptionService service;

    public PrescriptionController(PrescriptionService service) {
        this.service = service;
    }

    @GetMapping
    public List<Prescription> getAll() {
        return service.findAll();
```

```
    }

    @GetMapping("/{id}")
    public ResponseEntity<Prescription> getById(@PathVariable String id) {
        return service.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Prescription> create(@RequestBody Prescription rx) {
        Prescription saved = service.save(rx);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @PutMapping("/{id}")
    public Prescription update(
            @PathVariable String id,
            @RequestBody Prescription rx) {
        return service.update(id, rx);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable String id) {
        service.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```

Interview Questions

Q8: What is @RestController?

@RestController is a specialized version of @Controller that combines @Controller and @ResponseBody. It marks a class as a REST API controller where every method returns data (JSON/XML) directly instead of view names.

Q9: What is the difference between @PathVariable and @RequestParam?

@PathVariable extracts values from the URL path (e.g., /users/{id}). @RequestParam extracts values from query parameters (e.g., /users?id=123).

Q10: What is @RequestBody used for?

@RequestBody tells Spring to convert the incoming HTTP request body (usually JSON) into a Java object. Spring uses Jackson library for JSON-to-object conversion automatically.

Q11: What HTTP status code should you return when creating a new resource?

201 Created — indicates the resource was successfully created. You should also return the created resource in the response body.

Q12: What is the difference between PUT and POST?

POST creates a new resource. PUT updates an existing resource (replaces it completely). POST is not idempotent (calling twice creates two resources). PUT is idempotent (calling twice produces same result).

4. SPRING DATA JPA — DATABASE MADE EASY

What is JPA?

JPA (Java Persistence API) is a specification for mapping Java objects to database tables. Hibernate is the most popular JPA implementation.

Spring Data JPA sits on top of JPA and provides even more convenience.

4.1 Entity Class — Mapping to Database Table

An Entity represents a table in the database.

```
@Entity  // Marks this class as a database table
```

```
@Table(name = "prescriptions")  // Optional: customize table name
public class Prescription {

    @Id  // Primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY)  // Auto-increment
    private Long id;

    @Column(name = "prescription_id", unique = true, nullable = false)
    private String prescriptionId;

    @Column(name = "patient_name")
    private String patientName;

    private String drugName;  // Column name = drug_name (auto)

    private String status;

    @Column(name = "prescribed_date")
    private LocalDate date;

    // Constructors, getters, setters
}
```

Key Annotations:

- @Entity — Marks class as JPA entity (maps to table)
- @Table — Customize table name (optional)
- @Id — Marks primary key field
- @GeneratedValue — How primary key is generated (AUTO, IDENTITY, SEQUENCE)
- @Column — Customize column properties (name, length, nullable, unique)

4.2 Repository — Database Operations Without SQL

Spring Data JPA provides repositories that handle CRUD operations automatically.

```
public interface PrescriptionRepository extends JpaRepository<Prescription, Long> {
    // Spring generates implementation automatically!
    // Built-in methods:
    // - save(entity)
    // - findById(id)
    // - findAll()
    // - deleteById(id)
    // - count()
    // - existsById(id)
}
```

Using the repository in a service:

```
@Service
public class PrescriptionService {

    private final PrescriptionRepository repository;

    public PrescriptionService(PrescriptionRepository repository) {
        this.repository = repository;
    }

    public List<Prescription> findAll() {
        return repository.findAll();
    }

    public Optional<Prescription> findById(Long id) {
        return repository.findById(id);
    }

    public Prescription save(Prescription prescription) {
        return repository.save(prescription);
    }

    public void deleteById(Long id) {
```

```
        repository.deleteById(id);
    }
}
```

4.3 Custom Query Methods

Spring Data JPA can generate queries from method names automatically.

```
public interface PrescriptionRepository extends JpaRepository<Prescription, Long> {

    // Find by exact status
    List<Prescription> findByStatus(String status);

    // Find by patient name
    List<Prescription> findByPatientName(String name);

    // Find by status and date
    List<Prescription> findByStatusAndDate(String status, LocalDate date);

    // Find by patient name containing (LIKE query)
    List<Prescription> findByPatientNameContaining(String keyword);

    // Count prescriptions by status
    long countByStatus(String status);

    // Check if exists
    boolean existsByPrescriptionId(String prescriptionId);
}
```

How it works:

Spring reads the method name and generates SQL automatically:

- findByStatus → SELECT * FROM prescriptions WHERE status = ?
- findByPatientNameContaining → SELECT * WHERE patient_name LIKE %?%

4.4 @Query — Custom JPQL/SQL Queries

For complex queries, write custom JPQL (Java Persistence Query Language).

```
public interface PrescriptionRepository extends JpaRepository<Prescription, Long> {

    // JPQL query (uses entity names, not table names)
    @Query("SELECT p FROM Prescription p WHERE p.status = :status AND p.date
>= :fromDate")
    List<Prescription> findActiveAfterDate(
        @Param("status") String status,
        @Param("fromDate") LocalDate fromDate
    );

    // Native SQL query
    @Query(value = "SELECT * FROM prescriptions WHERE drug_name = ?1",
           nativeQuery = true)
    List<Prescription> findByDrugNative(String drugName);
}
```

4.5 Relationships — @OneToMany, @ManyToOne

Real-world: One Patient has many Prescriptions.

```
@Entity
public class Patient {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // One patient has many prescriptions
    @OneToMany(mappedBy = "patient", cascade = CascadeType.ALL)
    private List<Prescription> prescriptions;
```

```
}

@Entity
public class Prescription {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String drugName;

    // Many prescriptions belong to one patient
    @ManyToOne
    @JoinColumn(name = "patient_id")
    private Patient patient;
}
```

Interview Questions

Q13: What is JPA?

JPA (Java Persistence API) is a specification that defines how to map Java objects to relational database tables. It provides an abstraction layer for database operations. Hibernate is the most common implementation.

Q14: What is the difference between JPA and Hibernate?

JPA is a specification (interface). Hibernate is an implementation of that specification. Spring Data JPA uses Hibernate under the hood.

Q15: What does @Entity annotation do?

@Entity marks a Java class as a JPA entity, meaning it maps to a database table. Each instance represents a row in that table.

Q16: What is the difference between save() and saveAll() in JpaRepository?

save() saves a single entity. saveAll() saves a collection of entities in a batch, which is more efficient than calling save() multiple times.

Q17: How does Spring Data JPA generate queries from method names?

Spring Data JPA parses the method name and generates SQL based on keywords like findBy, countBy, deleteBy followed by property names. For example, findByStatusAndDate generates: SELECT * WHERE status = ? AND date = ?

Q18: What is the difference between JPQL and Native SQL?

JPQL uses entity and field names (Java objects), while Native SQL uses actual table and column names. JPQL is database-independent, Native SQL is database-specific but more powerful.

5. EXCEPTION HANDLING IN REST APIS

Why Exception Handling Matters

When something goes wrong (resource not found, validation failure), you want to return a clean, meaningful error message to the client, not a stack trace.

5.1 Custom Exceptions

Create custom exceptions for specific error scenarios.

```
public class PrescriptionNotFoundException extends RuntimeException {
    public PrescriptionNotFoundException(String id) {
        super("Prescription not found with ID: " + id);
    }
}
```

5.2 @ControllerAdvice — Global Exception Handler

Handles exceptions from all controllers in one place.

```
@ControllerAdvice  // Applies to all controllers
public class GlobalExceptionHandler {
```

```
    @ExceptionHandler(PrescriptionNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
            PrescriptionNotFoundException ex) {

        ErrorResponse error = new ErrorResponse(
            HttpStatus.NOT_FOUND.value(),
            ex.getMessage(),
            LocalDateTime.now()
        );

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenericException(
            Exception ex) {

        ErrorResponse error = new ErrorResponse(
            HttpStatus.INTERNAL_SERVER_ERROR.value(),
            "An error occurred: " + ex.getMessage(),
            LocalDateTime.now()
        );

        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(error);
    }
}
```

Error Response class:

```
public class ErrorResponse {
    private int status;
    private String message;
    private LocalDateTime timestamp;

    // Constructor, getters, setters
}
```

When exception is thrown, client receives:

```
{
    "status": 404,
    "message": "Prescription not found with ID: RX123",
    "timestamp": "2024-02-15T10:30:00"
}
```

Real-World CIGNA Example

```
@RestController
@RequestMapping("/api/prescriptions")
public class PrescriptionController {

    @Autowired
    private PrescriptionService service;

    @GetMapping("/{id}")
    public Prescription getById(@PathVariable String id) {
        // If not found, throws PrescriptionNotFoundException
        // GlobalExceptionHandler catches it and returns 404 with clean message
        return service.findById(id)
            .orElseThrow(() -> new PrescriptionNotFoundException(id));
    }
}
```

Interview Questions

Q19: What is @ControllerAdvice?

@ControllerAdvice is used to handle exceptions globally across all controllers. Instead of writing try-catch in every controller method, you centralize exception handling in one place.

Q20: What is @ExceptionHandler?

@ExceptionHandler is used inside @ControllerAdvice to handle specific exception types. When that exception is thrown anywhere in the application, this method handles it.

Q21: What is the difference between @ControllerAdvice and @RestControllerAdvice?

@RestControllerAdvice is a specialized version that combines @ControllerAdvice and @ResponseBody. Use it for REST APIs to return response bodies directly.

6. INPUT VALIDATION

Why Validation?

Never trust user input. Validate data before saving to database to prevent:

- Invalid data (empty names, negative prices)
- SQL injection attacks
- Data integrity issues

Bean Validation Annotations

```
@Entity
public class Prescription {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Prescription ID cannot be blank")
    private String prescriptionId;

    @NotBlank(message = "Patient name is required")
    @Size(min = 2, max = 100, message = "Name must be 2-100 characters")
    private String patientName;

    @NotBlank(message = "Drug name is required")
    private String drugName;

    @NotNull(message = "Date cannot be null")
    @PastOrPresent(message = "Date cannot be in future")
    private LocalDate date;

    @Email(message = "Invalid email format")
    private String email;
}
```

Common Validation Annotations:

- @NotNull — Field cannot be null
- @NotBlank — String cannot be null, empty, or whitespace
- @Size(min=, max=) — String/collection length
- @Min, @Max — Number range
- @Email — Valid email format
- @Pattern(regexp=) — Matches regex pattern
- @Past, @Future, @PastOrPresent — Date validations

Using @Valid in Controller

```
@RestController
@RequestMapping("/api/prescriptions")
public class PrescriptionController {

    @PostMapping
```

```
    public ResponseEntity<Prescription> create(
            @Valid @RequestBody Prescription prescription) {
        // @Valid triggers validation
        // If validation fails, returns 400 Bad Request automatically

        Prescription saved = service.save(prescription);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
}
```

When validation fails, Spring returns:

```
{
    "timestamp": "2024-02-15T10:30:00",
    "status": 400,
    "error": "Bad Request",
    "message": "Validation failed",
    "errors": [
        "patientName: Patient name is required",
        "drugName: Drug name is required"
    ]
}
```

Interview Questions

Q22: What is Bean Validation?

Bean Validation (JSR-303/JSR-380) is a Java specification for validating object properties using annotations like @NotNull, @Size, @Email. Spring Boot includes Hibernate Validator as the implementation.

Q23: What is @Valid used for?

@Valid triggers validation on the method parameter. When placed before @RequestBody, Spring validates the incoming object and returns 400 Bad Request if validation fails.

FINAL SUMMARY — WHAT YOU MUST KNOW

CORE CONCEPTS FOR INTERVIEWS

- IoC and Dependency Injection — How Spring manages objects
- Spring Boot auto-configuration — Reduces manual configuration
- @RestController, @GetMapping, @PostMapping — Building REST APIs
- @PathVariable vs @RequestParam — URL parameter extraction
- HTTP Status Codes — When to use 200, 201, 404, 500
- @Entity and JpaRepository — Database mapping and operations
- @ControllerAdvice — Global exception handling
- @Valid — Input validation

REAL-WORLD CIGNA CONNECTION

In interviews, connect theory to your CIGNA experience:

"In my CIGNA Pharmacy API project, we used Spring Boot to build RESTful services. For example, I developed a prescription management endpoint using @RestController where I implemented GET, POST, PUT, and DELETE operations. The controller used @PathVariable to extract prescription IDs from the URL and @RequestBody to accept prescription data as JSON. We used Spring Data JPA with JpaRepository for database operations, which automatically generated queries based on method names like findByStatus. For error handling, we implemented @ControllerAdvice to catch exceptions like PrescriptionNotFoundException and return clean 404 responses with custom error messages to the client."

NEXT STEP: BUILD THE STUDENT API

Now that you understand the theory, it's time to code. Start with the Student Management API project. You'll implement everything you just learned:

- Create Entity class with validation
- Build Repository interface
- Write Service layer with business logic
- Implement REST Controller with all CRUD endpoints

- Add global exception handling
- Test with Postman
- Push to GitHub

By building this, you'll be able to answer 90% of Spring Boot interview questions with confidence.